EXPERIMENT-1

Aim:

- Solving 2 water jug Problem: Write a program to solve the 2-water jug problem by manually providing moves from the user.
- Write a program to implement Missionaries and Cannibals problem.
  Constraints: three missionaries and three cannibals must cross a river using a boat which can carry at most two people, under the constraint that, for both banks, if there are missionaries present on the bank, they cannot be outnumbered by cannibals (if they were, the cannibals would eat the missionaries). The boat cannot cross the river by itself with no people on board.

Theory:

Water Jug Problem is a classical problem in AI, it demonstrates the concepts of state-space search and problem-solving using operators.

The problem is that we are given two jugs of fixed and known capacity and the user needs to find if they can measure a given quantity of water using only the two jugs. We have unlimited supply of water.

Allowed Operations:

- Fill Jug 1 to the brim.
- Fill Jug 2 to the brim.
- Empty Jug 1.
- Empty Jug 2.
- Pour water from Jug 1 to Jug 2.
- Pour water from Jug 2 to Jug 1.

Missionaries and Cannibals Problem also a classical problem in AI, it demonstrates the concept of constraint satisfaction and search problem, it shows how constraints affect problem solving and how to maintain valid states.

The problem is that we have three cannibals and three missionaries on the left bank of the river and they all want to cross the river, we have to select correct order of passage

between the river so as to allow everyone to reach the right bank safely, but they only have a single boat of maximum capacity of two people, obviously boat cannot move itself and requires at least one person to operate the boat, cannibals will eat the missionaries if theirs number is greater than that of missionaries

Algorithm:

Two Water Jug Problem:-

1. Start.
2. Define functions for all the opertions with proper constraints.
3. Define the main function with a list of two elements with value as zero, representing the water in the two jugs.
4. Create a menu and take input from user to select which operation to perform and then print the list representing the state of both the jugs, and call the main function.
5. Continue taking inputs from until goal state is reached or the user wishes to exit the program.
6. Stop.

Missionaries and Cannibals Problem:-

1. Start.
2. Initialise start and end state along with all valid moves.
3. Initialise a queue with the current start (i.e start) and a list containing all the visited states (i.e start).
4. Initialise a visited list containing all the visited states (i.e start).
5. Create a function to check the validity of the states.
6. Initialise a while loop, which terminates upon reaching the end_state or when queue becomes empty.
7. Inside the while loop, generate all the possible states by applying the moves list and if it is a valid state then append it into the queue and visited list.
8. When the goal state is reached the path is printed and the loop breaks or else loop breaks when the queue goes empty.
9. Stop.

Dataset Description:

Water Jug Problem:

Source:

There is no external dataset, the data is user-driven and system driven based on jug capacity, target amount and manual selection of operations.

Attributes:

- Jug X capacity (x)
- Jug Y capacity (y)
- Current water level in Jug X
- Current water level in Jug Y
- Target amount of water
- User-selected operation number

Size:

- Very small and dynamic
- The size changes interactively based on user actions

Missionaries and Cannibals Problem

Source:

The dataset is predefined and internally generated based on problem constraints.

Attributes:

- Number of missionaries on the left bank
- Number of cannibals on the left bank
- Boat position
- Valid state transitions
- Sequence of states

Size:

- Finite state space
- Only valid and safe states are explored using BFS

Implementation:

Two Water Jug Problem:-

```
# Function to fill jug X completely

def fill_x(x, wtr):

  wtr[0] = x

  print(wtr[0], " ", wtr[1])


# Function to fill jug Y completely

def fill_y(y, wtr):

  wtr[1] = y

  print(wtr[0], " ", wtr[1])


# Function to empty jug X

def empty_x(wtr):

  wtr[0] = 0

  print(wtr[0], " ", wtr[1])


# Function to empty jug Y

def empty_y(wtr):

  wtr[1] = 0

  print(wtr[0], " ", wtr[1])


# Function to pour water from jug X to jug Y

def x_y(x, y, wtr):

  if(y-wtr[1] >= wtr[0]):

    wtr[1] += wtr[0]
```

```
    wtr[0] = 0
  else:
    wtr[0] -= y-wtr[1]
    wtr[1] = y
  print(wtr[0], " ", wtr[1])

# Function to pour water from jug Y to jug X
def y_x(x, y, wtr):
  if(x-wtr[0] >= wtr[1]):
    wtr[0] += wtr[1]
    wtr[1] = 0
  else:
    wtr[1] -= x-wtr[0]
    wtr[0] = x
  print(wtr[0], " ", wtr[1])

# Main function to perform operations manually
def twoWaterJugs(x, y, target):
  wtr = [0, 0] # Initial state

  while(True):
    print("       ")
    print("1. Fill X")
    print("2. Fill Y")
    print("3. Empty X")
    print("4. Empty Y")
    print("5. Pour X in Y")
    print("6. Pour Y in X")
```

```
    print("7. Break")


    opt = int(input("Enter the operation: "))


    # Check if target is reached
    if(x == target or y == target or x+y == target):
      print("Target reached")
      break
    elif(opt == 1): fill_x(x, wtr)
    elif(opt == 2): fill_y(y, wtr)
    elif(opt == 3): empty_x(wtr)
    elif(opt == 4): empty_y(wtr)
    elif(opt == 5): x_y(x, y, wtr)
    elif(opt == 6): y_x(x, y, wtr)
    elif(opt == 7): break
```

Missionaries and Cannibals Problem:-

```
from collections import deque
# (missionary, cannibal, boat)
# boat = 1 means left bank
# boat= 0 means right bank
start = (3, 3, 0)
end = (0, 0, 1)


# Queue for BFS storing current state and path
q = deque([(start, [start])])
visited = [start]
```

```
# Possible boat moves
moves = [(1, 0), (0, 1), (1, 1), (2, 0), (0,2)]

# Function to check validity of a state
def check(x, y):
  if(x < 0 or x > 3 or y < 0 or y > 3): return False

  if(x != 0 and y > x): return False
  elif(3-x != 0 and (3-y) > (3-x)): return False

  return True

# BFS algorithm
while(q):
  (x, y, z), steps = q.popleft();

  # Check goal state
  if((x, y, z) == end):
    for i in steps:
      print(i)
    break

  # Try all possible moves
  for l, r in moves:
    if(z == 0): # Boat on right bank
      nz = 1
      nx = x - l
```

```
  ny = y - r
else: # Boat on left bank
  nz = 0
  nx = x + l
  ny = y + r

# Check if new state is valid and unvisited
if(check(nx, ny) == True):
  if((nx, ny, nz) not in visited):
    visited.append((nx, ny, nz))
    q.append(((nx, ny, nz), steps+[(nx, ny, nz)]))
```

Output:

Two Water Jug Problem:-

```
1 twoWaterJugs(3, 5, 4)
```

```
...
    1. Fill X
    2. Fill Y
    3. Empty X
    4. Empty Y
    5. Pour X in Y
    6. Pour Y in X
    7. Break
    Enter the operation: 2
    0   5

    1. Fill X
    2. Fill Y
    3. Empty X
    4. Empty Y
    5. Pour X in Y
    6. Pour Y in X
    7. Break
    Enter the operation: 6
    3   2

    1. Fill X
    2. Fill Y
    3. Empty X
    4. Empty Y
    5. Pour X in Y
    6. Pour Y in X
    7. Break
    Enter the operation: 3
    0   2
```

```
1. Fill X
2. Fill Y
3. Empty X
4. Empty Y
5. Pour X in Y
6. Pour Y in X
7. Break
Enter the operation: 6
2   0

1. Fill X
2. Fill Y
3. Empty X
4. Empty Y
5. Pour X in Y
6. Pour Y in X
7. Break
Enter the operation: 2
2   5

1. Fill X
2. Fill Y
3. Empty X
4. Empty Y
5. Pour X in Y
6. Pour Y in X
7. Break
Enter the operation: 6
3   4

1. Fill X
2. Fill Y
3. Empty X
4. Empty Y
5. Pour X in Y
6. Pour Y in X
7. Break
```

Missionaries and Cannibals Problem:-

```
...  (3, 3, 0)
     (2, 2, 1)
     (3, 2, 0)
     (3, 0, 1)
     (3, 1, 0)
     (1, 1, 1)
     (2, 2, 0)
     (0, 2, 1)
     (0, 3, 0)
     (0, 1, 1)
     (1, 1, 0)
     (0, 0, 1)
```

Result Analysis:

For Two Water Jug Problem, the user has to solve the problem by selecting appropriate operations, and once the target is reached the program exits or when the user wishes of exit.

In the Missionaries and Cannibals Problem, we have used the concepts of AI to solve the problem, all we are doing is just specifying all the constraints. All possible states are generated by the program in a bfs manner starting from the start state and are checked for validity. All valid states are stored in the queue and the entire path is printed once the end state is reached.

Conclusion:

These experiments helped us to understand how real-world problems can be broken down into well-defined states and how to solve them using a sequence of valid actions within the given constraints. They clearly demonstrate the importance of state representation, rule-based decision making, and goal checking in Artificial Intelligence.